

Wprowadzenie do egzaminu Systemy liczbowe I

Matura rozszerzona z informatyki – Sesja 1

Kacper Gębusia

- 1 O egzaminie: format, punktacja, strategia
- 2 System pozycyjny – teoria
- 3 Konwersje: dziesiętny $\leftrightarrow$ dwójkowy / ósemkowy / szesnastkowy (w tym schemat Hornera)
- 4 Szybkie konwersje binarny $\leftrightarrow$ hex/oct (grupowanie bitów)
- 5 Systemy o dowolnej podstawie
- 6 Implementacja w języku Python
- 7 Ćwiczenia

Jak wygląda egzamin?

- Poziom rozszerzony, **Formuła 2023**
- Jeden ciągły egzamin **przy komputerze**
- Czas: **210 minut**
- Punktacja: **50 punktów**

Dwa typy zadań:

- **Teoretyczne** – zamknięte, z luką, krótkiej i rozszerzonej odpowiedzi (zapis cyfrowy)
- **Praktyczne** – programowanie (Python/C++/Java), arkusz kalkulacyjny, baza danych (SQL/Access)

- Rozwiązania zadań praktycznych zapisujemy w folderze z numerem **PESEL**
- **Punkty częściowe** – nawet niepełne rozwiązanie może dać punkty
- Zawsze zostawiamy czytelny, uporządkowany zapis rozumowania
- Orientacyjny podział czasu:
 - ok. 60–70 min – zadania teoretyczne
 - ok. 140 min – zadania praktyczne
 - Planowanie działań na początku
 - Lepiej zaplanować zrobienie 90% i je wykonać niż iść po kolei i zrobić pół arkusza
- Wszystkie arkusze i zasady oceniania: cke.gov.pl

System pozycyjny – definicja

W systemie pozycyjnym o podstawie p liczbę zapisujemy jako ciąg cyfr $c_n, c_{n-1}, \dots, c_1, c_0$, a jej wartość to:

$$c_n \cdot p^n + c_{n-1} \cdot p^{n-1} + \dots + c_1 \cdot p^1 + c_0 \cdot p^0$$

Cyfry, których można użyć: $0, 1, \dots, p - 1$.

System	Podstawa	Cyfry	Zastosowanie
Dziesiętny	10	0–9	Codzienne obliczenia
Dwójkowy (binarny)	2	0, 1	Architektura komputera
Ósemkowy	8	0–7	Uprawnienia w Linuxie
Szesnastkowy	16	0–9, A, B, C, D, E, F	Adresy MAC, IPv6, kolory

Dlaczego komputery liczą binarnie?

- Najprościej i najpewniej reprezentować **dwa stany fizyczne**
- np. jest napięcie / nie ma napięcia
- System dwójkowy = naturalny „język” sprzętu komputerowego

Sumujemy wartości pozycji, na których jest cyfra 1.

Przykład 1: $101011_2 = ?_{10}$

$$1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 32 + 8 + 2 + 1 = 43$$

Schemat Hornera – szybsza metoda

Zamiast liczyć osobno każdą potęgę podstawy: zaczynamy od najstarszej cyfry, mnożymy przez podstawę i dodajemy kolejną cyfrę – aż do końca.

Dla liczby $c_n c_{n-1} \dots c_1 c_0$ w systemie o podstawie p :

$$(((\dots ((c_n \cdot p + c_{n-1}) \cdot p + c_{n-2}) \cdot p + \dots) \cdot p + c_1) \cdot p + c_0$$

Zalety: mniej mnożeń, nie trzeba liczyć wysokich potęg – najwygodniejsza metoda do liczenia na kartce, działa dla **dowolnej podstawy**.

Unikamy potęgowania podstawy. Wynik obliczamy iteracyjnie: mnożąc dotychczasowy wynik przez podstawę p i dodając kolejną cyfrę.

wynik = wynik * podstawa + cyfra

Schemat Hornera – przykłady

Przykład 1b: $101011_2 = ?_{10}$ (cyfry od najstarszej: 1,0,1,0,1,1)

$$1 \xrightarrow{\cdot 2 + 0} 2 \xrightarrow{\cdot 2 + 1} 5 \xrightarrow{\cdot 2 + 0} 10 \xrightarrow{\cdot 2 + 1} 21 \xrightarrow{\cdot 2 + 1} \mathbf{43}$$

Ten sam wynik co w Przykładzie 1 ✓

Przykład 1c (inna podstawa): $2456_7 = ?_{10}$ (cyfry: 2,4,5,6)

$$2 \xrightarrow{\cdot 7 + 4} 18 \xrightarrow{\cdot 7 + 5} 131 \xrightarrow{\cdot 7 + 6} \mathbf{923}$$

Dziesiętny $\rightarrow$ dwójkowy

Dzielimy kolejno przez 2, zapisujemy reszty, czytamy je **od dołu do góry**.

Przykład 2: $43_{10} = ?_2$

$$43/2 = 21 \text{ reszta } 1$$

$$21/2 = 10 \text{ reszta } 1$$

$$10/2 = 5 \text{ reszta } 0$$

$$5/2 = 2 \text{ reszta } 1$$

$$2/2 = 1 \text{ reszta } 0$$

$$1/2 = 0 \text{ reszta } 1$$

Czytając reszty od dołu: **101011** – zgodnie z Przykładem 1.

Dziesiętny $\rightarrow$ szesnastkowy / ósemkowy

Analogicznie – dzielimy przez 16 (szesnastkowy) lub 8 (ósemkowy).

Przykład 3: $187_{10} = ?_{16}$

$$187/16 = 11 \text{ reszta } 11 (B)$$

$$11/16 = 0 \text{ reszta } 11 (B)$$

Wynik: **BB**₁₆ (sprawdzenie: $11 \cdot 16 + 11 = 187 \checkmark$)

Przykład 4: $154_{10} = ?_8$

$$154/8 = 19 \text{ reszta } 2$$

$$19/8 = 2 \text{ reszta } 3$$

$$2/8 = 0 \text{ reszta } 2$$

Wynik: **232**₈ (sprawdzenie: $2 \cdot 64 + 3 \cdot 8 + 2 = 154 \checkmark$)

Grupowanie bitów – najważniejsza sztuczka

- $16 = 2^4 \Rightarrow$ grupujemy bity **po 4** (od prawej) $\rightarrow$ system szesnastkowy
- $8 = 2^3 \Rightarrow$ grupujemy bity **po 3** (od prawej) $\rightarrow$ system ósemkowy
- Niepełną skrajną lewą grupę dopełniamy zerami z lewej strony

Nie trzeba nic dzielić!

Przykłady grupowania

Przykład 5 (bin $\rightarrow$ hex): $10110110_2 = ?_{16}$

$$1011\ 0110 \rightarrow B\ 6 \rightarrow \mathbf{B6}_{16}$$

Przykład 6 (bin $\rightarrow$ oct): $10110110_2 = ?_8$

$$010\ 110\ 110 \rightarrow 2\ 6\ 6 \rightarrow \mathbf{266}_8$$

Przykład 7 (niepełna grupa): $111010110_2 = ?_{16}$

$$0001\ 1101\ 0110 \rightarrow 1\ D\ 6 \rightarrow \mathbf{1D6}_{16}$$

(sprawdzenie: $111010110_2 = 470_{10}$; $1D6_{16} = 1 \cdot 256 + 13 \cdot 16 + 6 = 470 \checkmark$)

Algorytm jest **identyczny** jak dla 2, 8, 10, 16 – to tylko zastosowanie ogólnej definicji systemu pozycyjnego.

- Dziesiętny $\rightarrow$ podstawa p : dzielimy kolejno przez p , czytamy reszty **od dołu**
- Podstawa $p \rightarrow$ dziesiętny: sumujemy $c_i \cdot p^i$ lub schemat Hornera
- Dla $p > 10$: brakujące cyfry zapisujemy literami (10=A, 11=B, ...) – jak w hex

Przykład 8 (dziesiętny $\rightarrow$ podstawa 5): $100_{10} = ?_5$

$$100/5 = 20 \text{ reszta } 0$$

$$20/5 = 4 \text{ reszta } 0$$

$$4/5 = 0 \text{ reszta } 4$$

Wynik: **400_5** (sprawdzenie: $4 \cdot 25 + 0 + 0 = 100 \checkmark$)

Przykład 9 (podstawa 7 $\rightarrow$ dziesiętny): $2456_7 = ?_{10}$

$$2 \cdot 7^3 + 4 \cdot 7^2 + 5 \cdot 7 + 6 = 686 + 196 + 35 + 6 = \mathbf{923}_{10}$$

Przykład – podstawa większa niż 10

Przykład 10: $300_{10} = ?_{20}$

$$300/20 = 15 \text{ reszta } 0$$

$$15/20 = 0 \text{ reszta } 15 (F)$$

Wynik: **F0**₂₀ (sprawdzenie: $15 \cdot 20 + 0 = 300 \checkmark$)

Implementacja w Pythonie: Horner - dowolna podstawa $\rightarrow 10$

```
def horner(liczba, podstawa):  
    wynik = 0  
  
    for cyfra in liczba:  
        wynik = wynik * podstawa + int(cyfra)  
  
    return wynik  
  
print(horner("1101", 2))  
print(horner("210", 3))
```

Implementacja w Pythonie: $10 \rightarrow$ dowolna podstawa (≤ 10)

```
def decToBase(liczba, podstawa):  
    cyfry = ""  
  
    while liczba > 0:  
        reszta = liczba % podstawa  
        cyfry += str(reszta)  
        liczba //= podstawa  
  
    return cyfry[::-1]  
  
print(decToBase(31, 2))  
print(decToBase(120, 7))
```

Implementacja w Pythonie: 10 → dowolna podstawa

Funkcja `decToBase(liczba, podstawa)` w dwóch wersjach uzupełniona o obsługę systemów o podstawie większej niż 10:

```
def decToBase2(liczba, podstawa):  
  
    ZNAKI= "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"  
    cyfry = ""  
  
    while liczba > 0:  
        cyfry += ZNAKI[liczba % podstawa]  
        liczba //= podstawa  
  
    return cyfry[::-1]  
  
print(decToBase2(31, 2))  
print(decToBase2(120, 7))
```

Wersja z użyciem ciągu znaków

```
def decToBase3(liczba, podstawa):  
  
    ZNAKI= "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"  
    cyfry = []  
  
    while liczba > 0:  
        cyfry.append(ZNAKI[liczba % podstawa])  
        liczba //= podstawa  
  
    return "".join(reversed(cyfry))  
  
print(decToBase2(31, 2))  
print(decToBase2(120, 7))
```

Wersja z użyciem listy

Wynik powyższych funkcji będzie identyczny dla tych samych argumentów

Ćwiczenia na zajęciach (1/2)

- ➊ Zamień na system dwójkowy:
a) 25 b) 60 c) 100 d) 255
- ➋ Zamień na system dziesiętny:
a) 1010 b) 111000 c) 100001 d) 1111111
- ➌ Zamień na system szesnastkowy:
a) 200 b) 4095

Ćwiczenia na zajęciach (2/2)

- 4 Zamień z dwójkowego na szesnastkowy (grupowanie):
a) 1101100 b) 100000000
- 5 Zamień z dwójkowego na ósemkowy (grupowanie):
a) 111010110
- 6 Zamień na system o podstawie 5:
a) 58 b) 275
- 7 Zamień na system dziesiętny:
a) 341_6 b) 1234_5
- 8 Zamień na system dziesiętny metodą Hornera:
a) 110101_2 b) 425_7

- ➊ Zamień na system dwójkowy:
a) 37 b) 128 c) 199 d) 500
- ➋ Zamień na system dziesiętny:
a) 11011 b) 101101 c) 11110000 d) 10000001
- ➌ Zamień na system szesnastkowy (dowolną metodą):
a) 2500 b) 65535

- 4 Zamień z systemu dwójkowego na szesnastkowy i ósemkowy:
110101011
- 5 Ile bitów potrzeba, aby zapisać liczbę dziesiętną 1000 w systemie dwójkowym?
Podaj jej zapis binarny.
- 6 Zamień 89 na system o podstawie 3 oraz na system o podstawie 6.
- 7 Zamień 532_8 na system dziesiętny, a następnie na system o podstawie 4.

Dziękuję za uwagę!